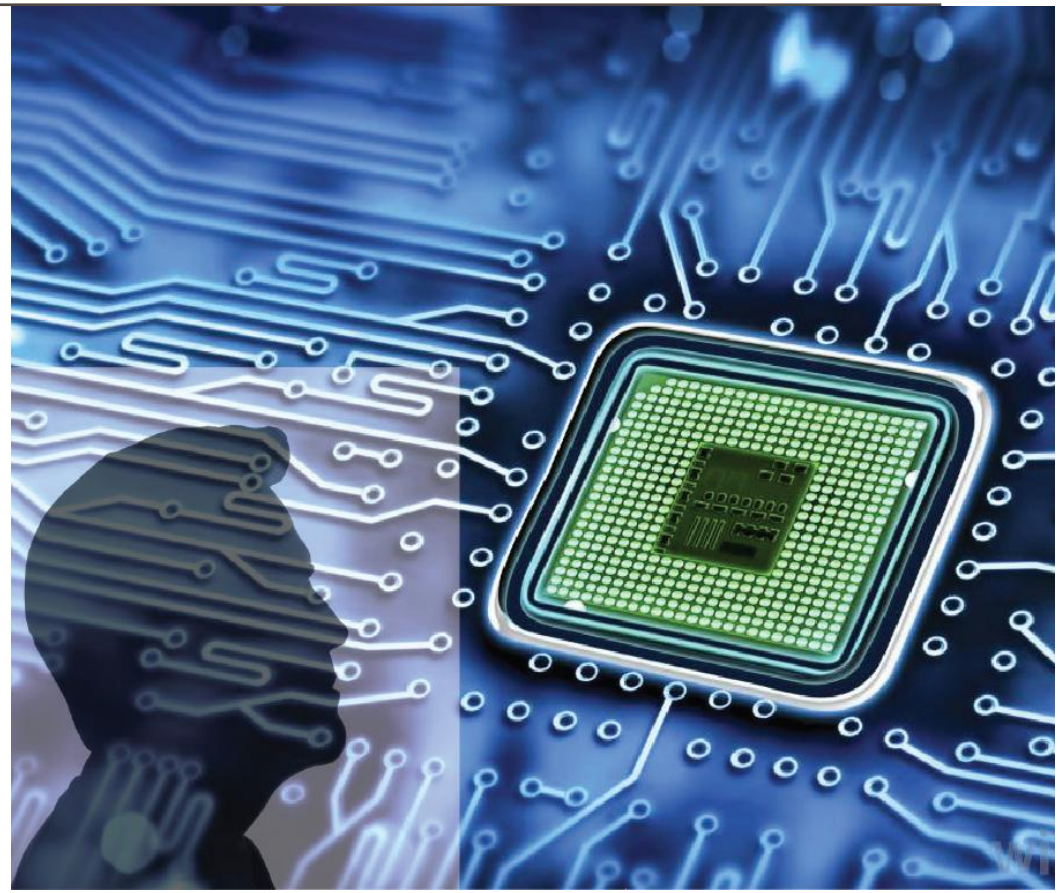
ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-2, Lecture-1
SystemC



Sneh Saurabh
19th August, 2025

SystemC

SystemC: Hello World Program

```
#include <systemc.h>

SC_MODULE(Hello_SystemC) { // declare module class

        SC_CTOR(Hello_SystemC) { // create a constructor
                SC_THREAD(module_thread); // register the process
        }

        void module_thread(void) { // define the process registered
                SC_REPORT_INFO("Module Thread Saying:", " Hello SystemC World!");
        }
};

int sc_main(int sc_argc, char* sc_argv[]) {

        //create an instance of the SystemC module
        Hello_SystemC HelloWorld_i("HelloWorld_i");

        sc_start(); // invoke the simulator


        return 0;
}
```

Code in a file main.cpp

Module Hello_SystemC:

- Constructor is defined
  - A thread named module_thread is registered
- Thread module_thread is defined
  - Prints “Hello …”

sc_main is the starting point:

- Instance named HelloWorld_i created
- Simulation started

Hello World: Compiling

Environment

- Using shell: tcsh (other shells can also be used)
- Go to the directory where code exists

Compilation

- Compiler: g++
- Include file path:
  - -I<path_of_include_files>
- Path of SystemC libraries:
  - -L<path_of_libraries>
- Name of SystemC library to link:
  - -lsystemc
- Source code: main.cpp
- Output: a.out

```
sneh@DESKTOP-C6Q06CF:~/test/systemC/hello_world$ tcsh
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ls
main.cpp  readme.txt
DESKTOP-C6Q06CF:~/test/systemC/hello_world> g++ -I/home/sne
h/systemC/systemc-master/include -L/home/sneh/systemC/syste
mc-master/lib-linux64/ -lsystemc main.cpp
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ls -lrt
total 36
-rw-r--r-- 1 sneh sneh   188 May 23 16:44 readme.txt
-rw-r--r-- 1 sneh sneh   508 Jul 21 13:28 main.cpp
-rwxr-xr-x 1 sneh sneh 26848 Jul 21 13:36 a.out
```

```
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ./a.out
./a.out: error while loading shared libraries: libsystemc-3
.0.0.so: cannot open shared object file: No such file or di
rectory
```

Shared Object/Library

Object file

- A file that contains machine code (and other data) generated by a compiler or assembler

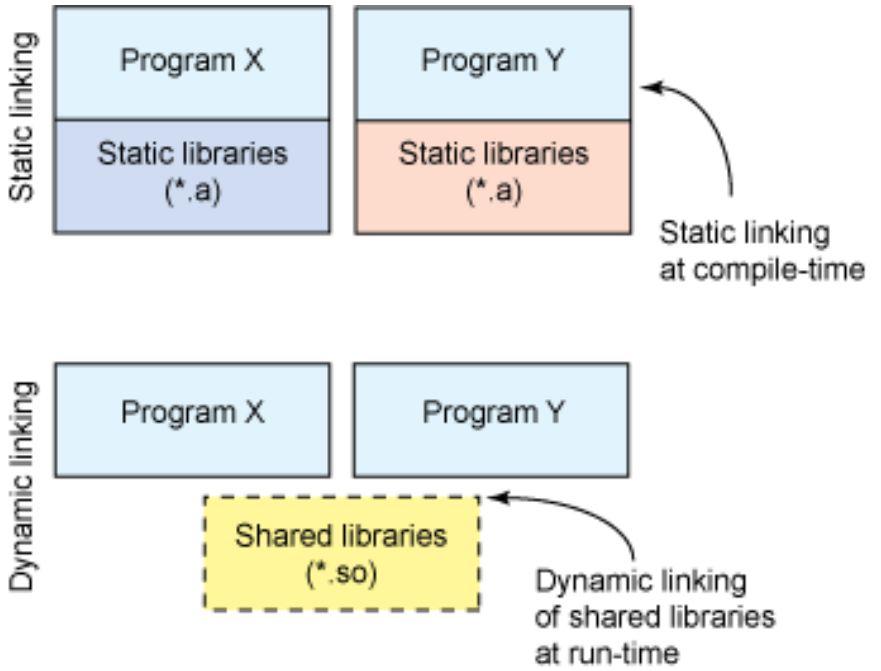
Static Linking

- Compilation can be done to include the code of the library in its executable file,
  - The library code embedded in the program's executable file is not usable by other programs.

Shared Object/Library

- Is a computer file that contains executable code designed to be used by multiple computer programs or other libraries at **runtime.**

- **Dynamic Linking**: Operating system loads the shared library from a file into memory at runtime



https://developer.ibm.com/tutorials/l-dynamic-libraries/

Hello World: Running

Running

- During compilation shared objects (.so) files were linked
- The functions inside these objects were not copied in the executable (a.out) and needs to be dynamically linked
- To load shared libraries dynamically
  - ➢ setenv LD_LIBRARY_PATH <path_of_libraries>

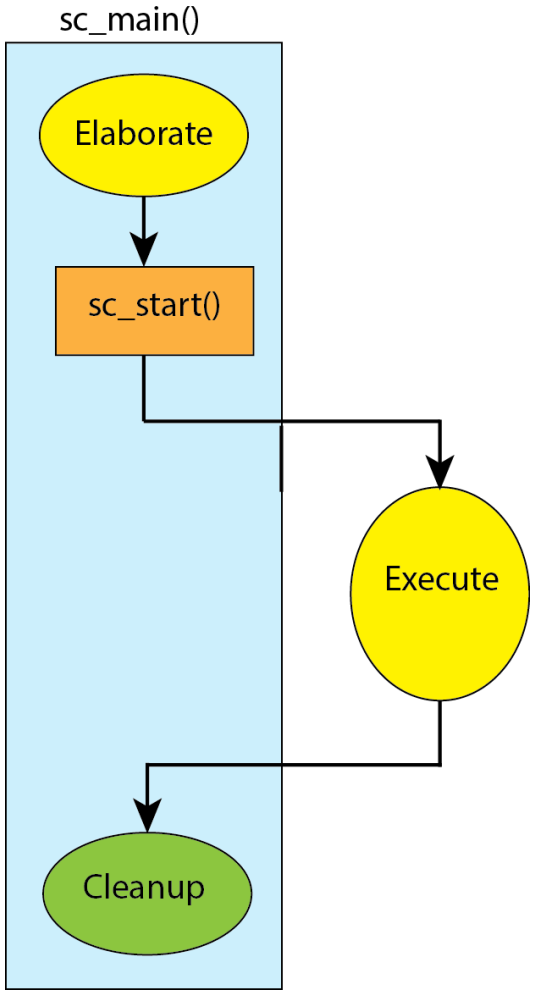
```
DESKTOP-C6Q06CF:~/test/systemC/hello_world> setenv LD_LIBRA
RY_PATH /home/sneh/systemC/systemc-master/lib-linux64/
```

```
DESKTOP-C6Q06CF:~/test/systemC/hello_world> ./a.out

        SystemC 3.0.0_pub_rev_20231124-Accellera --- Dec 19 2
 2023 10:52:02
        Copyright (c) 1996-2023 by all Contributors,
        ALL RIGHTS RESERVED

Info: Module Thread Saying::  Hello SystemC World!
DESKTOP-C6Q06CF:~/test/systemC/hello_world>
```

Output:

Info: Module Thread Saying:: Hello SystemC World!

Simulation: Phases



- Execution of SystemC simulation starts from sc_main

Phases of simulation

1. Elaboration
2. Execution
3. Cleanup

Elaboration

- Initialize data structures
- Establish module interconnection
- Constructors invoked for modules

Execution

- Starts with sc_start
- Performed by SystemC simulation kernel
- Runs various processes as per schedule (will be discussed later in detail)

Cleanup

- Post-processing after simulation is run

SystemC: Hello World Program

```
#include <systemc.h>

SC_MODULE(Hello_SystemC) { // declare module class

        SC_CTOR(Hello_SystemC) { // create a constructor
                SC_THREAD(module_thread); // register the process
        }

        void module_thread(void) { // define the process registered
                SC_REPORT_INFO("Module Thread Saying:", " Hello SystemC World!");
        }
};

int sc_main(int sc_argc, char* sc_argv[]) {

        //create an instance of the SystemC module
        Hello_SystemC HelloWorld_i("HelloWorld_i");

        sc_start(); // invoke the simulator


        return 0;
}
```

Code in a file main.cpp

Module Hello_SystemC:

- Constructor is defined
  - A thread named module_thread is registered
- Thread module_thread is defined
  - Prints “Hello …”

sc_main is the starting point:

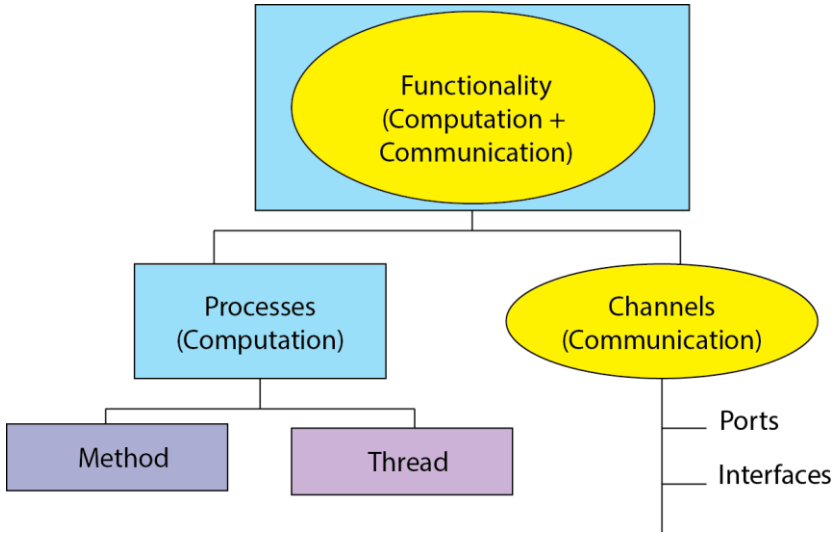
- Instance named HelloWorld_i created
- Simulation started

SystemC: Overview

- **Modules** are the basic building blocks of a SystemC design hierarchy

- Functionality can be separated into:
  - Computation
  - Communication

- Computational element: **process**
- Communication element: **channel**



- Two types of processes:
  - **Method**: starts and completes, entirely is stateless (values of non-static local variables lost on exit)
  - **Thread**: can yield (stop execution) to other processes by calling **wait**
- Processes can be dynamically triggered (in contrast to Verilog and VHDL)
  - Triggering event can be changed during runtime
- Events allow synchronization among processes

- Channels can be of many types: simple wire, FIFO, bus, etc.
- Modules have **ports/interfaces** for using the channel
  - Interface-based design style

SystemC: Hello World Program

```
#include <systemc.h>

SC_MODULE(Hello_SystemC) { // declare module class

        SC_CTOR(Hello_SystemC) { // create a constructor
                SC_THREAD(module_thread); // register the process
        }

        void module_thread(void) { // define the process registered
                SC_REPORT_INFO("Module Thread Saying:", " Hello SystemC World!");
        }
};

int sc_main(int sc_argc, char* sc_argv[]) {

        //create an instance of the SystemC module
        Hello_SystemC HelloWorld_i("HelloWorld_i");

        sc_start(); // invoke the simulator


        return 0;
}
```

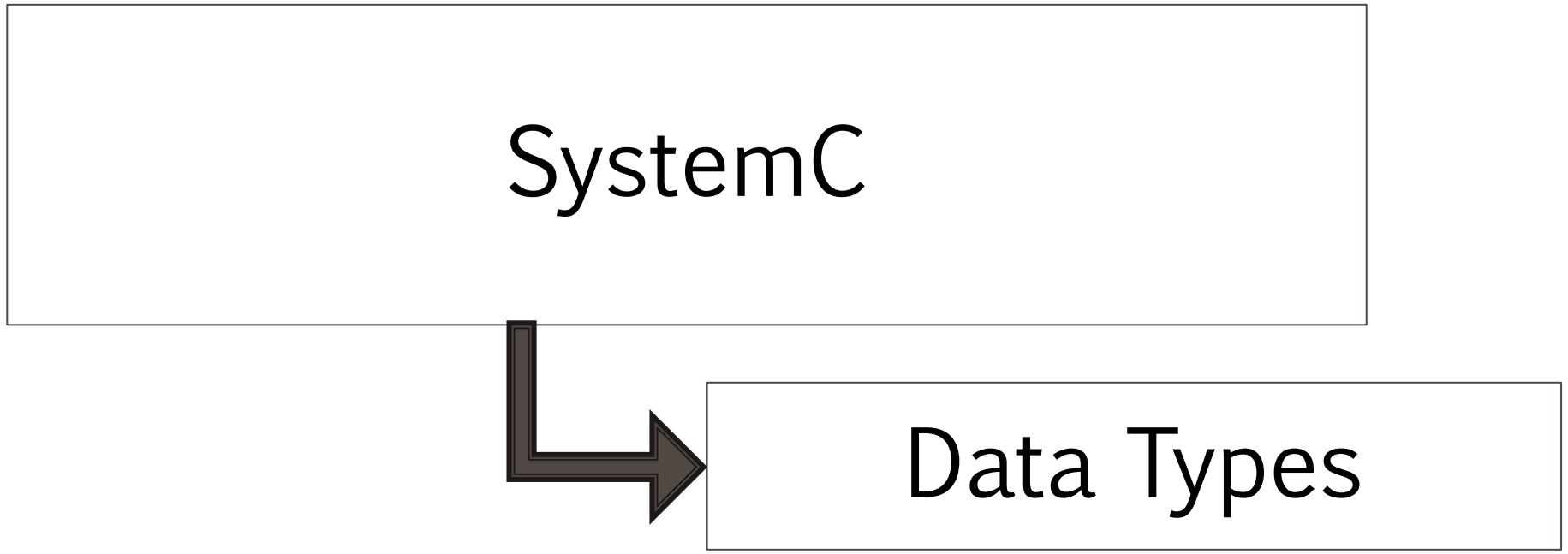
Code in a file main.cpp

Module Hello_SystemC:

- Constructor is defined
  - A thread named module_thread is registered
- Thread module_thread is defined
  - Prints “Hello …”

sc_main is the starting point:

- Instance named HelloWorld_i created
- Simulation started


SystemC
Data Types

Template in C++

- Pass the data type as a parameter so that we don't need to write the same code for different data types.

- C++ adds two new keywords to support templates: **template** and **typename**

```
template <typename T>
T myMax(T x, T y)
{
   return (x > y)? x: y;
}

int main()
{
   cout << myMax<int>(3, 7) << endl;
   cout << myMax<char>('g', 'e') << endl;
   return 0;
}
```

Compiler internally generates and adds below code

```
int myMax(int x, int y)
{
   return (x > y)? x: y;
}
```

Compiler internally generates and adds below code.

```
char myMax(char x, char y)
{
   return (x > y)? x: y;
}
```

```
SNEH2023:~/test/systemC/hello_world> ./a.out
7
g
```

Data Types: Overview

- All C++ datatypes are valid in SystemC
  - Example: char, short, int, long, float, double, bool, etc.
- Standard Template Library (STL) data types: such as string
- SystemC library supports various data types for logic and arithmetic operations

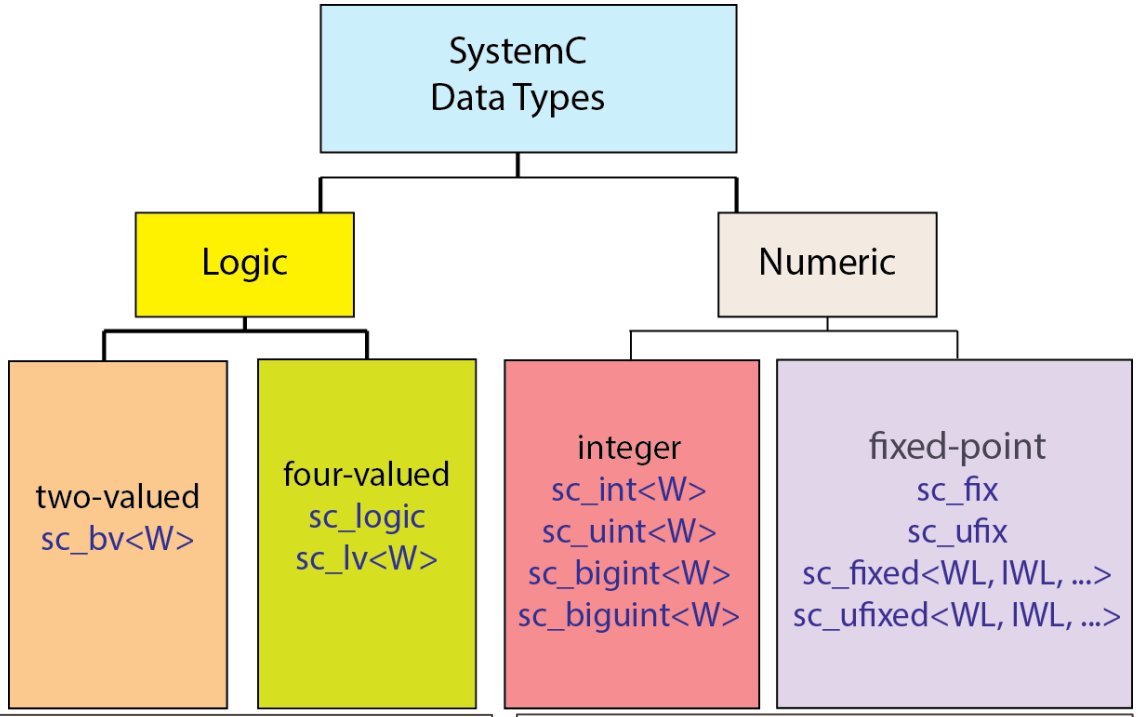
Two-value logic:

sc_bv<4> positions;

Four-value logic:

sc_lv<4> positions;

sc_logic flag;



Constants:

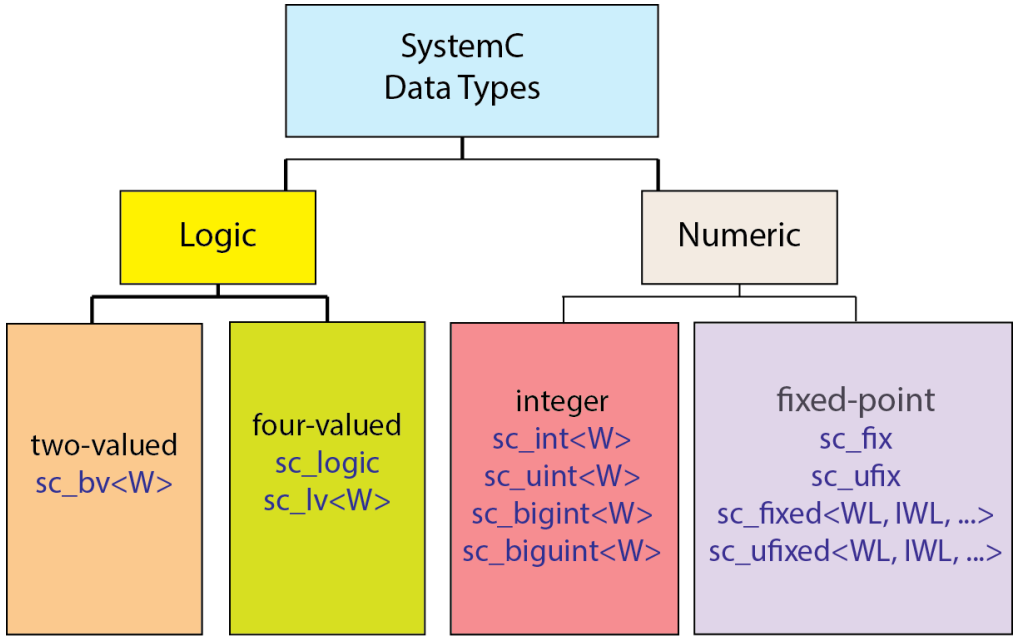
- logic 0 - SC_LOGIC_0, Log_0, or '0'
- logic 1 - SC_LOGIC_1, Log_1, or '1'

- High-impedance - SC_LOGIC_Z, Log_Z, 'Z' or 'z'
- Unknown - SC_LOGIC_X, Log_X, 'X' or 'x'

Data Types: Integers

Integers:

- The width of native C++ data types depends on the host processor and compiler
  - Typically, 8, 16, 32, or 64 bits in length and optimized/efficient for the host processor)
- SystemC integer data types can be of varied width
  - 1-64: sc_int and sc_uint
  - Greater than 64: sc_bigint, sc_biguint



Integers examples:

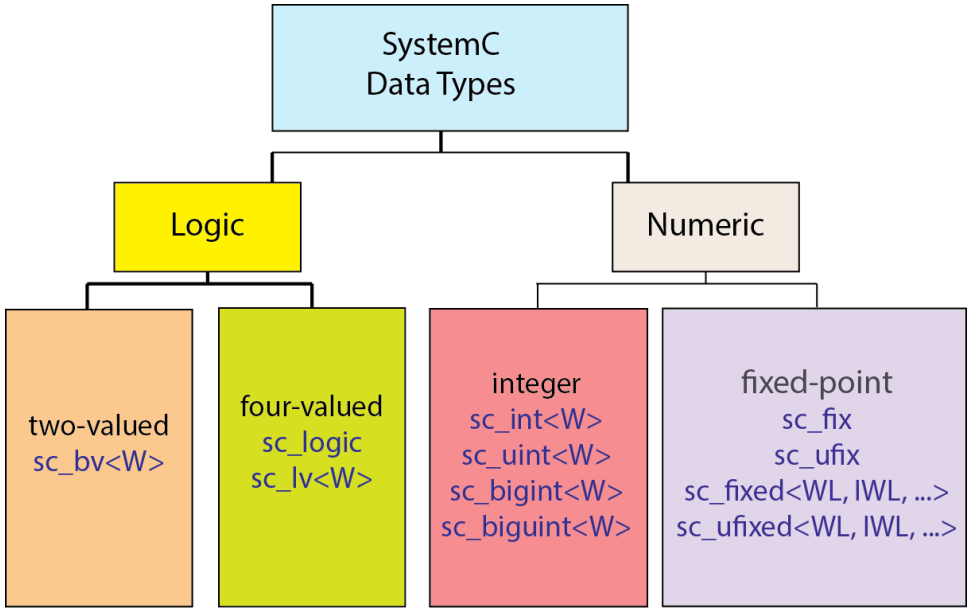
```
sc_int<3>  win_loss;
sc_uint<10>  index;
sc_biguint<72>  one_hot_state;
```

Data Types: Fixed Point (1)

Fixed-point:

- Can provide better fidelity for modeling the signal processing logic and synthesizable design
- In SystemC fixed-point representation is characterized by:
  - word length (total number of bits)
  - integer portion length.
  - overflow and quantization modes (optional)

- Fixed-point definitions are not in default SystemC include file
- Need to add extra definitions before including the SystemC include file




```
#define SC_INCLUDE_FX
#include <systemc>
sc_fixed<5,3>  weight;
```

Data Types: Fixed Point (2)

Types of Fixed-point:

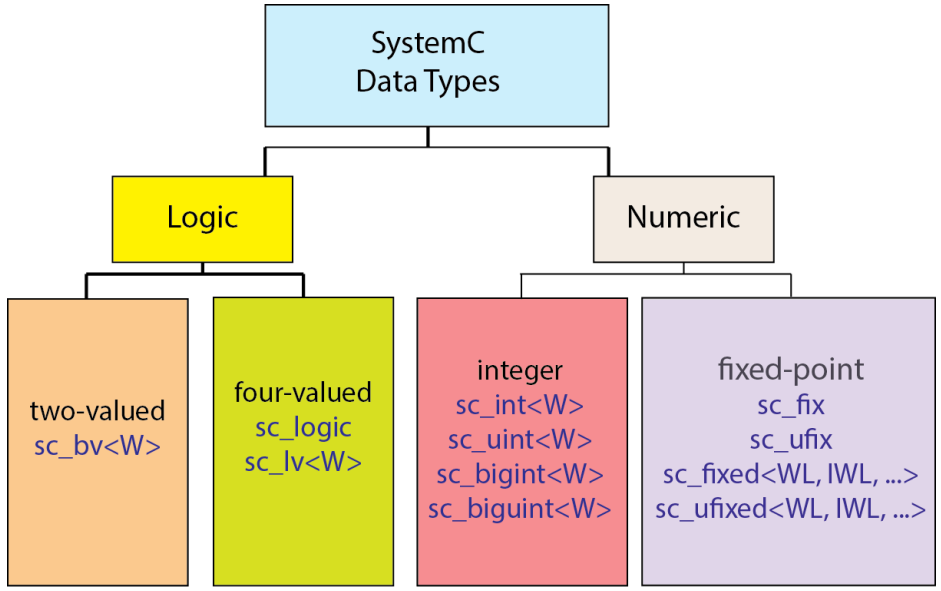
- Based on sign
  - Signed
  - Unsigned (added u in name)

```
sc_fixed<5,3>  weight;
sc_ufixed<8,6>  price;
```

How is the behavior of fixed-point defined

1. Run time using the constructor (without –ed)
2. Compile-time using template arguments (with -ed)

```
sc_fix weight(5,3);
sc_ufix price(sz, isz);
```



Fast fixed-point

- Implemented using C++ double and utilizes FPU for speed

```
sc_fix_fast bias(5,3);
sc_fixed_fast distance(8, 2);
```

Data Types: Operations (1)

- SystemC Data Types: namespace sc_dt

SystemC provides the following operations on its data types:

- Assignment:

```
sc_uint<4> I4 = 0;
I4 = 9;
```

- Initialization operations (with type conversions):

```
sc_int<3> i3 = 4;
```

```
i3= -4
```

- Equality:

```
sc_uint<3> i3 = 4;
sc_uint<3> j3 = 5;
bool result = (i3 == j3);
```

- Bitwise operations (single-bit and multi-bit select operations):

```
sc_uint<3> i3 = 4;
sc_uint<2> i2;
i3[1]= true;
i2 = i3.range(1,0);
```

```
i3= 4
i3= 6
i2= 2
```

- Conversions to C++ data types

```
sc_fixed<6,3> pi = 3.14;
double pied = pi.to_double();
string piedS = pi.to_string();
```

- Operator operator<< and operator>>

```
cout << pi;
```

Data Types: Operations (2)

- For numeric data types: arithmetic and relational operators provided

```
sc_fixed<6,3>  pi = 3.14;
sc_fixed<6,3>  r = 2.1;
sc_fixed<8,5>  area = pi*r*r;
bool big = (r <  2.43);
```

In general:

- SystemC data types are slower than native C++ data types
- More complex SystemC types are slower than simpler smaller types

Guidelines: Choose a data type that is close to the native C++ as far as possible

Data Types: Time

Definitions of time in simulation:

- **Wall-clock Time:** time from the start of execution to completion (including time waiting on other activities by the system)
- **Processor Time:** actual time spent executing the simulation (will be less than the simulation's wall-clock time)
- **Simulated Time:** time being modeled by the simulation (may be less than or greater than the simulation's wall-clock time)

- Data type sc_time is used to track **simulated time** and to specify **delays** and **timeouts**
  - Units can be: SC_SEC, SC_MS, SC_US, SC_NS, SC_PS, and SC_FS.
- sc_time can be used as operands for assignment, arithmetic, and comparison operations
- Current simulation time: sc_time_stamp()

```
sc_time startTime(50, SC_NS);
sc_time endTime(1, SC_US);
sc_time duration = endTime - startTime;
cout << "Start: " << startTime <<
  " End: " << endTime <<
  " Duration: " << duration <<
  " Current Time: " << sc_time_stamp() <<
  endl;
```

```
Start: 50 ns End: 1 us Duration: 950 ns Current Time: 0 s
```

References

- Black, David C., and Jack Donovan, eds. SystemC: From the ground up. Boston, MA: Springer US, 2004.